

PRACTICAL WEEK 5

Task:-1

Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

Source code:-

```
GNU nano 8.7.1 producer_consumer.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

#define SIZE 1000

int main()
{
    int pipefd[2];
    int data[SIZE];
    pid_t pid;

    struct timespec start, end;

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        exit(1);
    }

    clock_gettime(CLOCK_MONOTONIC, &start);

    pid = fork();
    if (pid < 0)
    {
        perror("fork");
        exit(1);
    }
    if (pid > 0)
```

```
GNU nano 8.7.1 producer_consumer.c *
    wait(NULL);

    clock_gettime(CLOCK_MONOTONIC, &end);

    double time_taken =
        (end.tv_sec - start.tv_sec) * 1000000.0 +
        (end.tv_nsec - start.tv_nsec) / 1000.0;

    printf("\nProducer: Generated %d integers.\n", SIZE);
    printf("Communication time: %.2f microseconds\n", time_taken);
    printf("Data transferred: %lu bytes\n", sizeof(data));
    printf("Communication efficiency: %.2f bytes/microsecond\n",
        sizeof(data) / time_taken);
}
else
{
    // Child - Consumer
    close(pipefd[1]);

    int received[SIZE];

    read(pipefd[0], received, sizeof(received));

    close(pipefd[0]);

    printf("Consumer: Received %d integers.\n", SIZE);
    printf("First value: %d\n", received[0]);
    printf("Last value: %d\n", received[SIZE - 1]);
}

return 0;
}
```

OUTPUT:-

```

avulasanjana@SanjanaReddy:~$ nano producer_consumer.c
avulasanjana@SanjanaReddy:~$ gcc producer_consumer.c -o producer_consumer
avulasanjana@SanjanaReddy:~$ ./producer_consumer
Consumer: Received 1000 integers.
First value: 1
Last value: 1000

Producer: Generated 1000 integers.
Communication time: 563.66 microseconds
Data transferred: 4000 bytes
Communication efficiency: 7.10 bytes/microsecond
avulasanjana@SanjanaReddy:~$ |

```

Observation

The parent process acts as the **producer** and sends data through an anonymous pipe. The child process acts as the **consumer** and reads the data from the pipe. The communication time and data transfer rate are measured to evaluate the efficiency of inter-process communication.

Task:-2

Develop a program that executes the equivalent of the shell command:

`ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and

`exec()` system calls.

Source code:-

```

GNU nano 8.7.1 pipe_exec.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    pid_t pid1, pid2;

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        exit(1);
    }

    // First child: ls -l
    pid1 = fork();

    if (pid1 < 0)
    {
        perror("fork");
        exit(1);
    }

    if (pid1 == 0)
    {
        // Redirect stdout to pipe
        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[0]);
        close(pipefd[1]);
    }
}

```

```

if (pid2 < 0)
{
    perror("fork");
    exit(1);
}

if (pid2 == 0)
{
    // Redirect stdin from pipe
    dup2(pipefd[0], STDIN_FILENO);

    close(pipefd[0]);
    close(pipefd[1]);

    execlp("grep", "grep", ".c", NULL);

    perror("execlp grep");
    exit(1);
}

// Parent closes pipe
close(pipefd[0]);
close(pipefd[1]);

waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);

printf("\nCommand executed: ls -l | grep \".c\"\n");

return 0;
}

```

OUTPUT

```

avulasanjana@SanjanaReddy:~$ nano pipe_exec.c
avulasanjana@SanjanaReddy:~$ gcc pipe_exec.c -o pipe_exec
avulasanjana@SanjanaReddy:~$ ./pipe_exec
-rwxr-xr-x 1 avulasanjana avulasanjana 16528 Aug 11 06:59 command_executor
-rw-r--r-- 1 avulasanjana avulasanjana 1387 Aug 11 06:59 command_executor.c
drwxr-xr-x 2 avulasanjana avulasanjana 4096 Aug 5 03:41 docs
-rwxr-xr-x 1 avulasanjana avulasanjana 16216 Aug 7 05:19 file_copy
-rw-r--r-- 1 avulasanjana avulasanjana 1113 Aug 7 05:17 file_copy.c
-rwxr-xr-x 1 avulasanjana avulasanjana 16432 Aug 13 08:28 fork_exec
-rw-r--r-- 1 avulasanjana avulasanjana 947 Aug 13 08:28 fork_exec.c
drwxr-xr-x 2 avulasanjana avulasanjana 4096 Aug 5 03:41 include
-rw-r--r-- 1 avulasanjana avulasanjana 1803 Aug 7 05:39 keyboard_input.c
-rw-r--r-- 1 avulasanjana avulasanjana 800 Aug 7 05:09 main_loop.c
-rw-r--r-- 1 avulasanjana avulasanjana 3934 Aug 11 06:41 parser_double_quot
es.c
-rw-r--r-- 1 avulasanjana avulasanjana 2249 Aug 11 06:34 parser_quotes.c
-rwxr-xr-x 1 avulasanjana avulasanjana 16344 Sep 10 08:33 pipe_exec
-rw-r--r-- 1 avulasanjana avulasanjana 1159 Sep 10 08:33 pipe_exec.c
-rwxr-xr-x 1 avulasanjana avulasanjana 15952 Aug 1 04:49 process
-rw-r--r-- 1 avulasanjana avulasanjana 80 Aug 1 04:48 process.c
-rw-r--r-- 1 avulasanjana avulasanjana 80 Aug 1 04:46 process.c.save
-rwxr-xr-x 1 avulasanjana avulasanjana 16400 Sep 10 08:31 producer_consumer
-rw-r--r-- 1 avulasanjana avulasanjana 1566 Sep 10 08:30 producer_consumer.
c
drwxr-xr-x 2 avulasanjana avulasanjana 4096 Aug 5 03:41 scripts
-rw-r--r-- 1 avulasanjana avulasanjana 559 Aug 13 12:04 session2_task1.c
-rw-r--r-- 1 avulasanjana avulasanjana 537 Aug 13 12:10 session2_task2.c
-rw-r--r-- 1 avulasanjana avulasanjana 4035 Aug 15 12:22 session3_task1.c
-rw-r--r-- 1 avulasanjana avulasanjana 2160 Aug 15 12:26 session3_task2.c
-rw-r--r-- 1 avulasanjana avulasanjana 1279 Aug 15 12:34 session4_task1.c
-rw-r--r-- 1 avulasanjana avulasanjana 2324 Aug 15 12:37 session4_task2.c
-rw-r--r-- 1 avulasanjana avulasanjana 1603 Aug 15 12:42 session5_task1.c
-rw-r--r-- 1 avulasanjana avulasanjana 3214 Aug 15 12:53 session5_task2.c
-rw-r--r-- 1 avulasanjana avulasanjana 1191 Aug 15 12:57 session6_task1.c
-rw-r--r-- 1 avulasanjana avulasanjana 1005 Aug 15 12:59 session6_task2.c
-rw-r--r-- 1 avulasanjana avulasanjana 1283 Aug 15 13:07 session7_task1.c
-rw-r--r-- 1 avulasanjana avulasanjana 1480 Aug 15 13:11 session7_task2.c
-rw-r--r-- 1 avulasanjana avulasanjana 3240 Aug 15 13:18 session8_task1.c
-rw-r--r-- 1 avulasanjana avulasanjana 3362 Aug 15 13:21 session8_task2.c
-rw-r--r-- 1 avulasanjana avulasanjana 879 Aug 13 10:13 skill_week1_task2.
c

```

```
-rw-r--r-- 1 avulasanjana avulasanjana 53 Aug 16 15:21 source.txt
drwxr-xr-x 2 avulasanjana avulasanjana 4096 Aug 5 03:41 src
-rw-r--r-- 1 avulasanjana avulasanjana 1261 Aug 16 14:34 week2pt1.c
-rw-r--r-- 1 avulasanjana avulasanjana 1233 Aug 16 15:21 week2pt2.c
-rw-r--r-- 1 avulasanjana avulasanjana 1020 Aug 16 15:38 week3pt1.c
-rw-r--r-- 1 avulasanjana avulasanjana 382 Aug 16 15:41 week3pt2.c
-rw-r--r-- 1 avulasanjana avulasanjana 1448 Aug 20 08:27 week4pt1.c
-rw-r--r-- 1 avulasanjana avulasanjana 858 Aug 20 08:33 week4pt2.c
```

```
Command executed: ls -l | grep ".c"
avulasanjana@SanjanaReddy:~$
```

Observation

The program successfully implements the shell pipeline `ls -l | grep ".c"` using Linux system calls. The first child executes `ls -l` and sends its output through the pipe. The second child receives the output through the pipe and executes `grep ".c"` to display C-related files.